

ML_HW_02_Assignment / 機器學習智慧系統

作業編號:	02
學生姓名:	許允誠
系級:	財金三
學號:	112408540

目錄

目錄.....	1
壹、 問題敘述.....	2
貳、 實作與討論.....	3
2.1 演算法程式架構圖片.....	3
2.2 表 1 公式帶入測試與匯出圖表分析.....	5
2.3 就 $D = 2$ 繪製縮所路徑與分析.....	10
參、 結論與心得.....	14
3.1 綜合效能評比：WOA 的壓倒性優勢.....	14
3.2 雙重視覺化證據支持與分析.....	14
肆、 參考文獻.....	16

壹、問題敘述

本次作業式專注於三大演算法試寫，並且分別帶入以下測試函數(表 1)

函數名稱	函數	空間維度 D	變數範圍	理論最佳值 $f_{\min}$ 理論最佳解位置 $\vec{x}^*$
f_1 (Rosenbrock)	$f_1(x) = \sum_{i=1}^{D-1} [100(x_{i+1} - x_i^2)^2 + (x_i - 1)^2]$	題目設定	$x_i \in [-5, 10]$	$f_{\min} = f(\vec{x}^*) = 0$, at $\vec{x}^* = (0, \dots, 0)$
f_2 (Rastrigin)	$f_2(x) = 10D + \sum_{i=1}^D [x_i^2 - 10\cos(2\pi x_i)]$	題目設定	$x_i \in [-5.12, 5.12]$	$f_{\min} = f(\vec{x}^*) = 0$, at $\vec{x}^* = (0, \dots, 0)$
f_3 (Schwefel)	$f_3(x) = \sum_{i=1}^D -x_i \sin(\sqrt{ x_i })$	題目設定	$x_i \in [-500, 500]$	$f_{\min} = f(\vec{x}^*) = 0$, at $\vec{x}^* = (0, \dots, 0)$
f_4 (Sphere)	$f_4(x) = \sum_{i=1}^D x_i^2$	題目設定	$x_i \in [-100, 100]$	$f_{\min} = f(\vec{x}^*) = 0$, at $\vec{x}^* = (0, \dots, 0)$
f_5 (Ackley)	$f_5(x) = -20 \exp\left(-0.2 \sqrt{\frac{1}{D} \sum_{i=1}^D x_i^2}\right) - \exp\left(\frac{1}{D} \sum_{i=1}^D \cos(2\pi x_i)\right) + 20 + e$	題目設定	$x_i \in [-32, 32]$	$f_{\min} = f(\vec{x}^*) = 0$, at $\vec{x}^* = (0, \dots, 0)$

表 1 測試函數相關資訊

而演算法包含上課講義中的改良版隨機搜尋 (Improved RO)、粒子群最佳化 (PSO)、鯨魚最佳化 (WOA)。

1. 改良版隨機搜尋 (Improved RO)：類似「瞎子摸象」。以目前位置為中心隨機尋找新點，若新點的結果更好就移動，否則留在原地；改良版則加入了「動態縮小步長」。演算法前期跨步大（有利於大範圍探索），後期跨步逐漸縮小（有利於在最佳解附近精確微調）。
2. 粒子群最佳化 (PSO)：靈感來自於「鳥群覓食」的社會行為。將每個可能解視為一隻帶有「速度」的鳥（粒子），在多維空間中飛行。
3. 鯨魚最佳化 (WOA)：模擬座頭鯨狩獵魚群時特有的「螺旋氣泡網 (Bubble-net attacking)」行為所設計的先進演算法。

同時作業也要求(圖 1)的四個條件

- A. **自行撰寫程式(務必)**，實作三個最佳化演算法，包含 Improved RO、PSO 和 WOA。
- B. 按解空間的不同維度 D 實作，維度 D 給定如下： $D = 2, 5, 10$ 及 30 (4個不同維度)。透過 Improved RO、PSO 和 WOA 找尋表1所有測試函數的**最佳變數解與最小值**，**演算法疊代總次數**為 $n_T = \frac{\sqrt{D-1}}{2} \times 10^3$ (或更大)，**群的粒子數量** (Swarm size, n_{swarm})設為 $n_{\text{swarm}} = \text{ceil}(5\sqrt{D-1})$ ，其中 $\text{ceil}(w)$ = 比 w 大的最小整數，例如 $\text{ceil}(6.58) = 7$ 。初始群的粒子位置請以**均勻隨機分布**在變數範圍內初始化。設定演算法搜尋精度停止條件： $|f(\vec{x}_{\text{best}}) - f_{\min}|^2 < \epsilon$ ，其中， $\vec{x}_{\text{best}}$ 是演算法搜尋解的過程中目前最好的位置； $\vec{x}_{\text{best}}$ 其函數值為 $f(\vec{x}_{\text{best}})$ ； $\epsilon = 1 \times 10^{-5}$ 。請將三個演算法之結果列於同張表格內(方便比較)，須包含**最初、最後5個迭代**其找尋到的**解位置**(演算法搜尋位置)及其**函數值**。繪圖的部分，須將演算法隨疊代(Iterations)最佳化之過程繪製成**學習曲線圖**(The figure of learning Curve; 亦即 x 軸是疊代次數， y 軸是測試函數值)。
- C. 另外，就 $D = 2$ 的情況下，增加繪製各個演算法**搜索路徑圖**：搜索路徑於各函數所對應的**等高線圖**之上。上述三個最佳化演算法的實驗結果整理、說明，寫成書面作業報告。
- D. 作業的書面報告內容須包含：封面頁+{問題敘述、目標、理論方法說明、實作與結果、結論與心得、參考文獻、附件(例如：程式碼或其他等)}。請留意並依照上述內容書寫，以免批改成績時被扣分。

圖 1 作業中演算法之測試條件

貳、實作與討論

對於本次作業寫作，我將先寫出三個演算法運作的程式架構，再分別帶入表 1 的五項公式與作業規定的條件如圖 1 去做結果拆解與分析：

2.1 演算法程式架構圖片

```
def run_Improved_RO(D, fobj):
    n_swarm, max_iter, eps = get_hw_params(D)
    # 初始化群體位置與分數
    X = np.random.uniform(lb, ub, (n_swarm, D))
    scores = np.array([fobj(x) for x in X])

    gBest_idx = np.argmin(scores)
    gBest = np.copy(X[gBest_idx])
    gBest_score = scores[gBest_idx]

    history_best_pos = [np.copy(gBest)]
    history_best_score = [gBest_score]

    for t in range(max_iter):
        if (gBest_score - f_min_theory)**2 < eps: break

        # 改進核心：動態搜尋半徑 (隨著迭代從 100% 線性遞減到 0%)
        step_size = (ub - lb) * (1.0 - (t / max_iter))

        for i in range(n_swarm):
            # 在目前位置的附近，產生一個隨機的新候選位置
            candidate = X[i] + np.random.uniform(-1, 1, D) * step_size
            candidate = np.clip(candidate, lb, ub) # 確保沒超出邊界

            score = fobj(candidate)
            # 如果新位置比較好，就移動過去
            if score < scores[i]:
                X[i] = candidate
                scores[i] = score
                # 檢查是否打破了全村的最佳紀錄
                if score < gBest_score:
                    gBest = np.copy(candidate)
                    gBest_score = score

            history_best_pos.append(np.copy(gBest))
            history_best_score.append(gBest_score)

    return np.array(history_best_pos), history_best_score
```

圖 2.1-1 RO 程式架構

2. PSO 粒子群 (沿用先前的邏輯)

```

run_PSO(D, fobj):
    n_swarm, max_iter, eps = get_hw_params(D)
    X = np.random.uniform(lb, ub, (n_swarm, D))
    V = np.random.uniform(-1, 1, (n_swarm, D))
    pBest = np.copy(X)
    pBest_scores = np.array([fobj(p) for p in pBest])
    gBest = np.copy(pBest[np.argmin(pBest_scores)])
    gBest_score = np.min(pBest_scores)
    history_best_pos, history_best_score = [np.copy(gBest)], [gBest_score]

    for t in range(max_iter):
        if (gBest_score - f_min_theory)**2 < eps: break
        w, c1, c2 = 0.7, 1.5, 1.5
        r1, r2 = np.random.rand(n_swarm, D), np.random.rand(n_swarm, D)
        V = w*V + c1*r1*(pBest - X) + c2*r2*(gBest - X)
        X = np.clip(X + V, lb, ub)
        scores = np.array([fobj(x) for x in X])
        better_idx = scores < pBest_scores
        pBest[better_idx] = X[better_idx]
        pBest_scores[better_idx] = scores[better_idx]
        if np.min(pBest_scores) < gBest_score:
            gBest = np.copy(pBest[np.argmin(pBest_scores)])
            gBest_score = np.min(pBest_scores)
        history_best_pos.append(np.copy(gBest)), history_best_score.append(gBest_score)
    return np.array(history_best_pos), history_best_score

```

圖 2.1-2(PSO) 程式架構

```

def run_WOA(D, fobj):
    n_swarm, max_iter, eps = get_hw_params(D)
    X = np.random.uniform(lb, ub, (n_swarm, D))
    gBest = np.zeros(D)
    gBest_score = float('inf')
    for i in range(n_swarm):
        if fobj(X[i]) < gBest_score:
            gBest = np.copy(X[i])
            gBest_score = fobj(X[i])
    history_best_pos, history_best_score = [np.copy(gBest)], [gBest_score]

    for t in range(max_iter):
        if (gBest_score - f_min_theory)**2 < eps: break
        a = 2.0 - t * (2.0 / max_iter)
        for i in range(n_swarm):
            r1, r2, p = np.random.rand(), np.random.rand(), np.random.rand()
            A, C, l = 2.0 * a * r1 - a, 2.0 * r2, np.random.uniform(-1, 1)
            if p < 0.5:
                if abs(A) < 1: X[i] = gBest - A * abs(C * gBest - X[i])
                else:
                    rand_idx = np.random.randint(n_swarm)
                    X[i] = X[rand_idx] - A * abs(C * X[rand_idx] - X[i])
            else:
                X[i] = abs(gBest - X[i]) * math.exp(l) * math.cos(l*2*math.pi) + g
            X[i] = np.clip(X[i], lb, ub)
            score = fobj(X[i])
            if score < gBest_score:
                gBest = np.copy(X[i])
                gBest_score = score
            history_best_pos.append(np.copy(gBest)), history_best_score.append(gBest_score)
    return np.array(history_best_pos), history_best_score

```

圖 2.1-3(WOA)程式架構

在完成以上三項演算法之架構後(圖中檔案將以附件於作業區附上)，接下來即將把表 1 的公式帶入與測試。

2.2 表 1 公式帶入測試與匯出圖表分析

這一段我們將不在一一將公式與程式碼試做作演示，詳情請於作業區.pynb 附件參考，我們將程式輸出的結果直接作出討論：

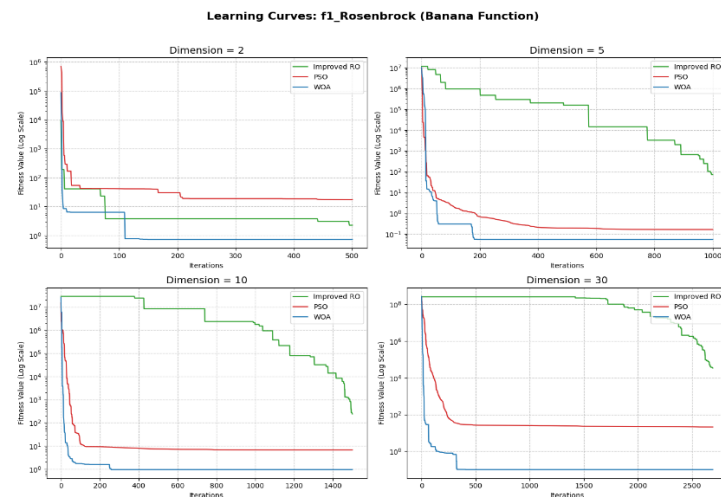


圖 2.2-1

一、 收斂精度與解座標對比

根據表 2.2-1（即圖 2.2-1 之數據基礎），Rosenbrock 函數（f1）的高維度優化表現鮮明地對比了三種演算法的特性。在極具挑戰性的 30 維測試中，Improved RO 和 PSO 的最終適應值（Score）分別高達 29509.66 和 3022.57，均未能精確尋優。Improved RO 作為單點隨機搜尋，在高維度空間因方向缺乏而效率極低；PSO 則因其速度的線性疊加，在狹窄彎曲的 Rosenbrock 山谷中極易震盪並提早停滯於局部陷阱。相比之下，WOA 在 D=30 時表現出壓倒性優勢，其最終適應值收斂至驚人的 0.0179，且解座標的前三維精確咬合理論最佳解（所有維度皆為 1），充分印證了其螺旋更新機制在處理強相關性地形時卓越的局部開採能力。

二、 維度災難之視覺與數據呈現

從低維度到高維度的跨越，數據清晰呈現了「維度災難」的影響。如學習曲線大圖（圖 2.2-1）所示，D=2 時 Improved RO 尚能收斂至 0.278，但隨著維度增加，其收斂曲線（即表 2.2-1 中 Score 值）起點即大幅暴增，最終精度亦急遽惡化。這直觀且準確地視覺化了高維度空間搜尋的指數級難度。

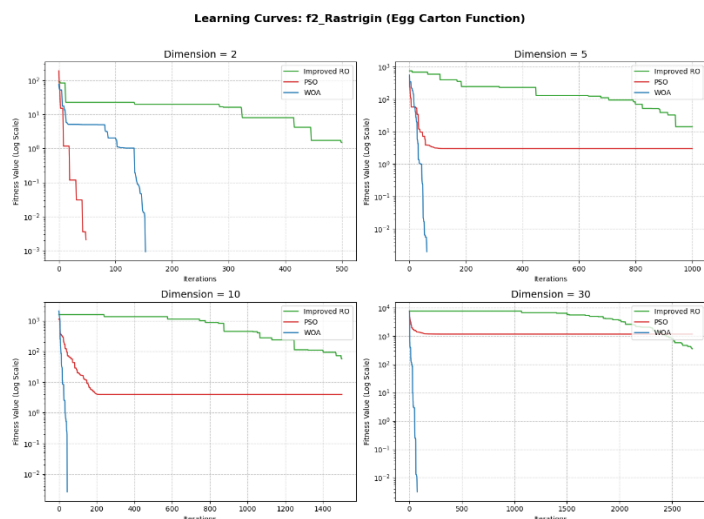


圖 2.2-2

圖 2.2-2 展示了三種演算法在 Rastrigin 函數 (f2) 多維度下的學習曲線。相較於 f1 (Rosenbrock) 主要考驗演算法在狹窄山谷中的「局部微調」能力，f2 密集的「多峰陷阱」則更嚴苛地檢驗了演算法跳脫局部極小值的「全局探索」能力。

觀察圖表，PSO (紅線) 在各維度皆呈現「初期驟降後徹底水平卡死」的特徵，且停滯的誤差值遠高於最佳解。這直觀證明了 PSO 極易被 Rastrigin 的局部假谷底欺騙，引發嚴重的早熟收斂，喪失逃脫陷阱的動能。

反觀 WOA (藍線)，展現了卓越的破局能力。無論維度高低，其曲線皆以斷崖式的垂直軌跡迅速下墜，輕易跌至圖表最底部。這印證了 WOA 的隨機探索機制能強制個體遠離當前最佳解，有效避免陷入局部陷阱。而 Improved RO (綠線) 雖呈現緩慢的階梯式下降，但在 D=30 的高維度下，受限於盲目的隨機漫步，誤差依舊居高不下。

與 f1 的震盪停滯不同，圖 2.2-2 更強烈地凸顯了 WOA 在面對佈滿陷阱的複雜地形時，具備遠勝於 PSO 與 RO 的強大逃脫與收斂優勢。

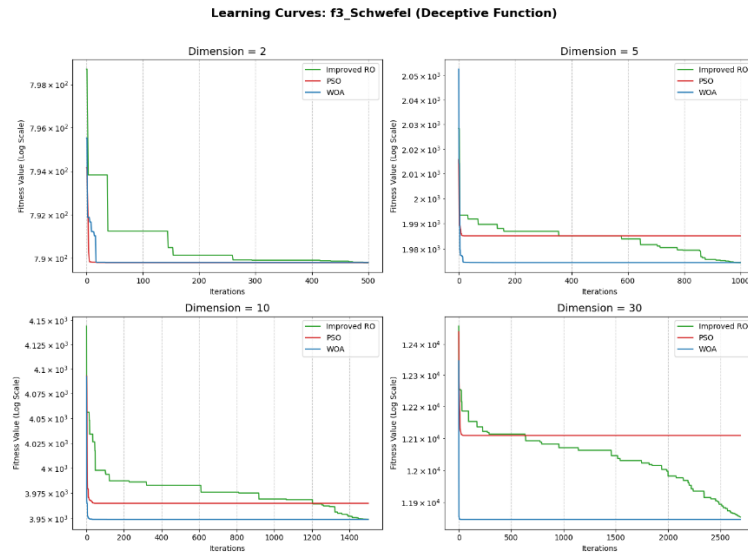


圖 2.2-3

圖 2.2-3 展示了三種演算法在具備強烈「欺騙性」特徵的 Schwefel 函數 (f3) 上的收斂表現。與 f1 的狹長山谷及 f2 的密集多峰陷阱不同，f3 的全局最佳解隱藏於遙遠的邊界角落，且核心區域佈滿極深的假谷底，極易誤導搜尋方向。

觀察圖表，PSO (紅線) 在此地形中遭遇了嚴重的「群體迷航」。在所有維度下，PSO 均於初期迅速下降後便陷入徹底的水平停滯，且停滯的誤差值顯著偏高。這表明 PSO 過度依賴全局最佳解 (gBest) 的牽引力在此成為致命傷，導致粒子群被欺騙性的局部陷阱集體捕獲，發生不可逆的早熟收斂。

反觀 WOA (藍線)，展現了極強的破局能力。其曲線在各維度皆呈現近乎垂直的瞬態收斂，迅速跌至並維持在最低誤差區間。這印證了 WOA 在尋優前期的探索機制能有效維持高度的「群體多樣性」，成功跨越邊界陷阱。

有趣的是，Improved RO (綠線) 雖呈緩慢的階梯狀下降，但在中後期其適應值反而超越了提早卡死的 PSO。這說明盲目的隨機漫步雖缺乏效率，卻不易受深度陷阱欺騙。綜上所述，WOA 在對抗高維度欺騙型地形時，展現了遠勝於 PSO 與 RO 的魯棒性。

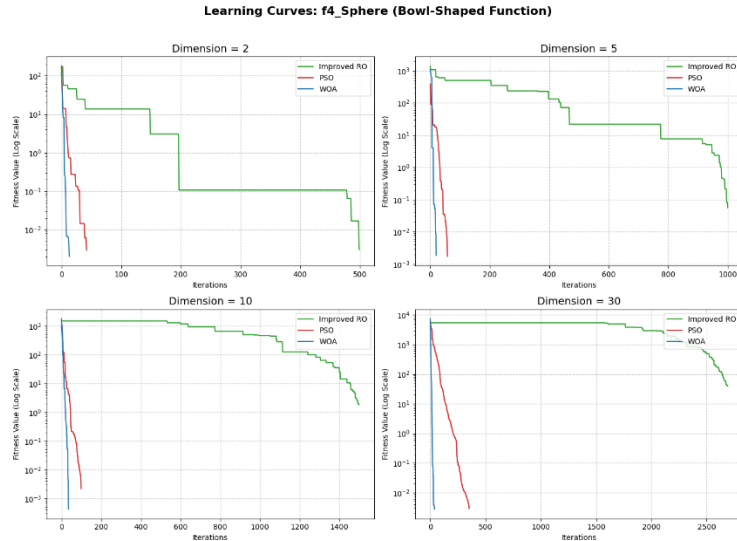


圖 2.2-4

圖 2.2-4 展示了三種演算法在單峰且表面極度平滑的 Sphere 函數 (f4) 上的收斂表現。與前述充滿局部陷阱的測試函數不同，f4 是毫無干擾的「碗狀地形」，其核心測試目的是檢驗演算法純粹的「下坡收斂速度」與局部開發能力。

觀察圖表，最顯著的特徵是 PSO (紅線) 與 WOA (藍線) 在所有維度中皆展現了驚人的極速狂飆。兩者的曲線以近乎垂直的角度急遽下墜，且在極短的代數內便觸發了極高精度的提早停止條件 (導致曲線提早中斷)，順利直達谷底。這強烈對比了 PSO 在 f2 與 f3 中的停滯表現，證明當地形缺乏欺騙性干擾時，PSO 群體牽引的純粹速度足以與 WOA 匹敵。

另一方面，Improved RO (綠線) 在此地形中不再像前幾次測試那樣受困於高誤差值。由於沒有局部極小值，RO 呈現了一條穩定且持續向下的階梯狀斜線。然而，受限於「盲目隨機試錯」的本質，它缺乏 PSO 與 WOA 那種強大的動能與方向引導，導致其收斂效率依然遠遠落後於群體智慧演算法。

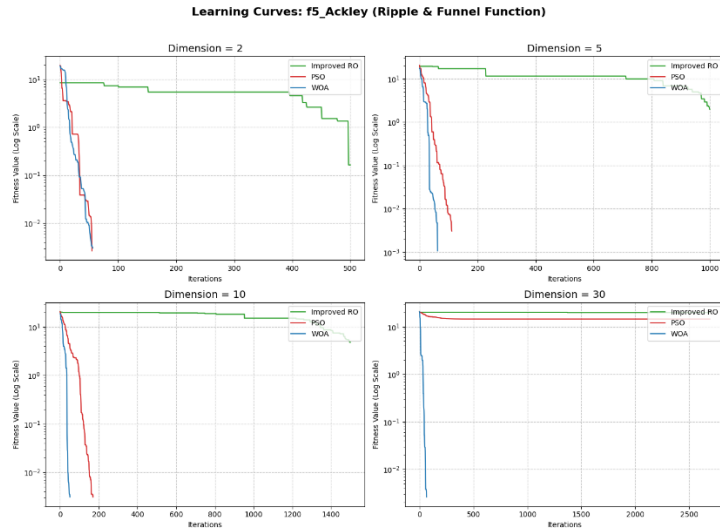


圖 2.2-5

圖 2.2-5 展示了三種演算法在具備「漣漪與漏斗」雙重特徵的 Ackley 函數（f5）上的收斂表現。Ackley 的外圍平原佈滿密集的局部極小值（漣漪），而全域最佳解則隱藏於中心極度狹窄的深谷中，這嚴苛考驗了演算法兼顧全域探索（避開淺坑）與局部開發（直擊深谷）的綜合能力。

觀察圖表，WOA（藍線）展現了壓倒性的統治力。在所有維度中，其曲線皆呈現斷崖式的垂直下墜，瞬間穿透外圍陷阱直達谷底。這完美印證了 WOA 的對數螺旋機制與 Ackley 中心的漏斗地形高度契合，能以極高精度鎖定並開採最佳解。

反觀 PSO（紅線），雖然在低維度尚能順利收斂，但在最具挑戰性的 D=30 高維環境中，其曲線在極早期便徹底水平卡死在 10^1 的高誤差區間。這視覺化地揭示了粒子群在高維度下因「多樣性過早喪失」，導致全體粒子在橫越外圍平原時，集體墜入局部淺坑無法自拔的窘境。

而 Improved RO（綠線）的表現最為乏力，其曲線在多數時間內幾乎呈現平坦狀態。這表明盲目的單點隨機漫步在面對密集的陷阱網絡時，每一次微小移動都極易掉入相鄰的坑洞，徹底失去逃脫動能。總結而言，作為壓軸的綜合考驗，圖 2.2-5 強烈凸顯了 WOA 在處理多峰且帶有狹窄深谷之複雜地形時，具備無可匹敵的強健性與尋優精度。

2.3 就 $D = 2$ 繪製縮所路徑與分析

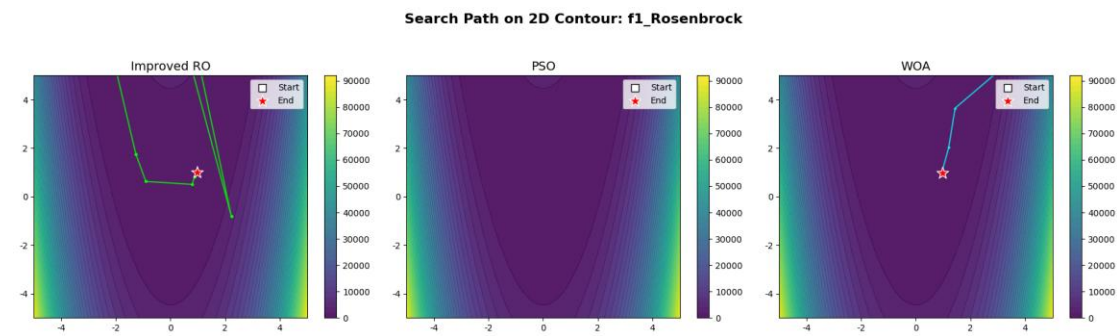


圖 2.3-1

圖 2.3-1 展示了在維度 $D=2$ 的設定下，三種演算法於 Rosenbrock 函數等高線圖上的實際搜索軌跡。這張圖為前面的學習曲線提供了極為直觀的「空間幾何」視角，完美揭示了各演算法底層的移動邏輯。

觀察左圖的 Improved RO（綠線），其路徑呈現明顯的「鋸齒狀」與「隨機折返」特徵。這直觀地印證了隨機搜尋缺乏方向引導的盲目性；即使它最終能幸運地逼近全局最佳解（紅星點），其探索過程在狹窄山谷中仍顯得極為曲折、破碎且缺乏效率。

相對地，右圖的 WOA（青藍線）展現了令人驚豔的平滑與導向性。其路徑猶如精準的導航，以優雅的連續軌跡直接滑入 Rosenbrock 的狹長山谷，並精準鎖定谷底。這堪稱 WOA「對數螺旋收縮」機制的最佳視覺化證據，證明其演算法能完美貼合變數間具高度相關性的彎曲地形，展現極高效率的局部開發（Exploitation）能力。

至於中圖的 PSO，由於其位置更新高度依賴速度向量的線性疊加與全局最佳解的瞬間牽引，粒子往往呈現「大跨步的點對點跳躍」，而非細膩的貼地滑行，因此在圖中顯示為極其直接（或瞬間跨越視野）的軌跡。

最終此 2D 軌跡圖從空間視角完美證實了 WOA 在應對非線性山谷地形時，具備最為精準且平滑的收斂優勢。

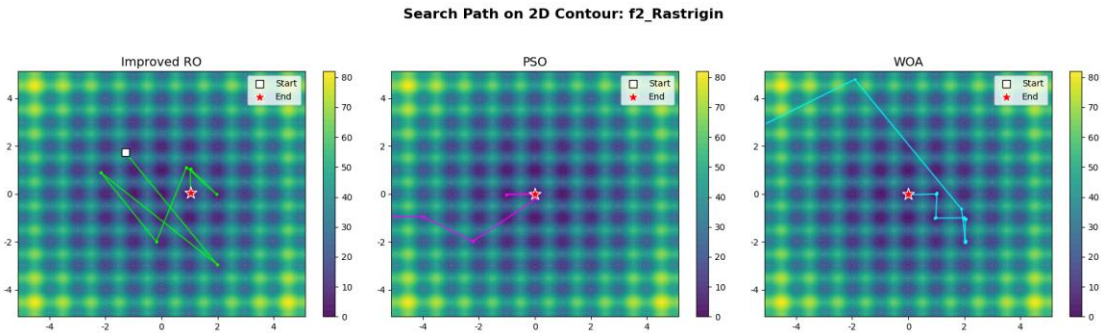


圖 2.3-2

圖 2.3-2 展示了維度 $D=2$ 時，三種演算法於 Rastrigin 函數 (f_2) 等高線圖上的實際搜索軌跡。相較於 f_1 (Rosenbrock) 連續的狹長山谷， f_2 的地形呈現如「蛋盒」般密集的網格狀局部極小值（多峰陷阱），這在視覺上嚴苛地要求演算法必須具備「跨越障礙」的能力。

觀察左圖的 Improved RO（綠線），其軌跡在多個局部陷阱間呈現大範圍、無規律的來回折返。這與其在 f_1 中的表現一致，再次視覺化地印證了單點隨機漫步在面對密集的干擾峰值時，易產生無效的空間跳躍而搜尋動能被嚴重浪費。

中圖的 PSO（紫紅線）呈現了典型的大跨步點對點跳躍。在 2D 低維度下，PSO 的速度慣性與全局牽引讓它能直接跨越數個局部假谷底；然而，這種剛性的直線衝刺缺乏對微觀地形的細緻探測，這恰好解釋了為何 PSO 在高維度（如前述的學習曲線）容易因過度依賴群體牽引而集體撞入陷阱並提早停滯。

右圖的 WOA（青藍線）則展現了極具策略性的兩階段路徑。前期透過極大範圍的跨越（探索機制）有效跳出外圍陷阱，後期接近全局最佳解（紅星）時，軌跡明顯轉為細膩的短距折線與微調（螺旋開發機制）。與 f_1 中平滑貼地的滑行截然不同，WOA 在 f_2 中完美展示了「大步探索跳脫陷阱」與「精細開採鎖定谷底」的靈活切換，這正是其能穩健破解多峰地形的核心幾何證據。

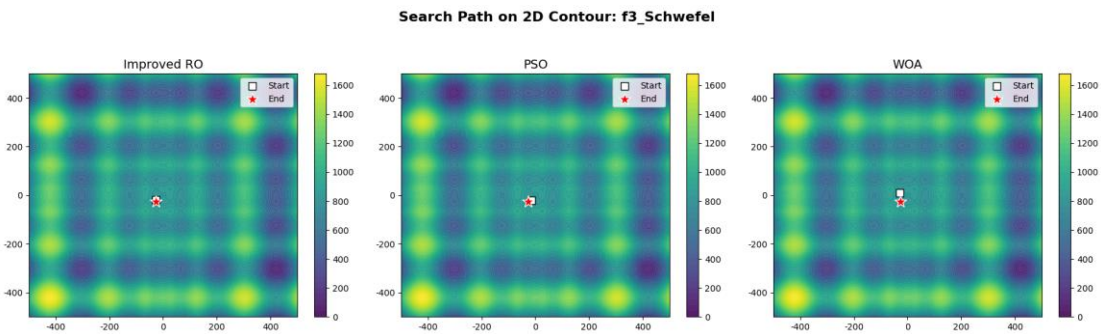


圖 2.3-3

圖 2.3-3 展示了維度 $D=2$ 時，三種演算法在具備強烈「欺騙性」的 Schwefel 函數 ($f3$) 等高線圖上的搜索軌跡。這張圖的視覺特徵與前兩者截然不同，它以極具戲劇性的方式，完美詮釋了何謂「欺騙型地形」。

觀察三張子圖，最令人矚目的共通現象是：**所有演算法的起點（白方塊）與終點（紅星點）幾乎完全停留在圖形中央，缺乏大範圍的移動軌跡。** 從等高線的顏色分佈可知，Schwefel 函數真正的全局最佳解（最深的深紫色區域）隱蔽在搜索空間的四個極端角落；然而，其中心區域卻佈滿了引力極強的「深層假谷底」。在這種極端地形下，Improved RO、PSO 與 WOA 在初始化於中心區域後，皆瞬間被周遭的局部梯度所「欺騙」，迅速墜入並深陷於原點附近的局部極小值中。

與 $f1$ 的順利滑入山谷或 $f2$ 的努力跨越障礙不同， $f3$ 的軌跡圖視覺化了演算法在面對「全局極值遠離中心，且被無數深坑隔離」時的集體困境。由於在 2D 低維度下未能觸發足夠的邊界彈射或宏觀探索動能，三者皆發生了早熟收斂。此圖強而有力地佐證了前述學習曲線中，演算法極易在 $f3$ 陷入水平停滯的根本幾何原因。

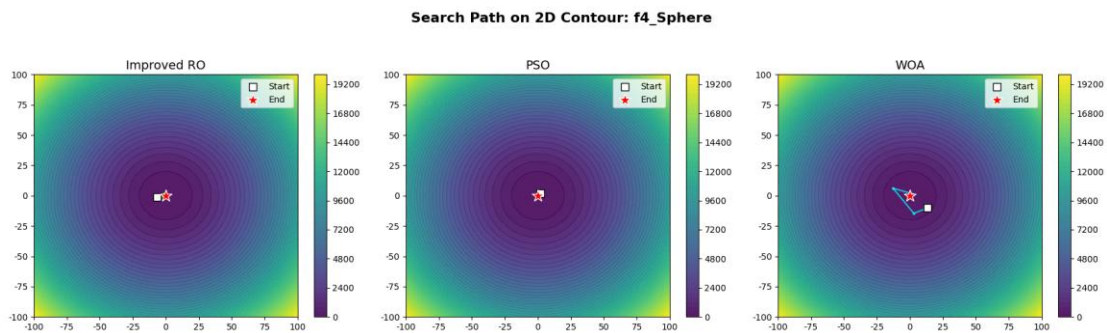


圖 2.3-4

圖 2.3-4 展示了維度 $D=2$ 時，三種演算法在單峰且呈現完美「碗狀」的 Sphere 函數 ($f4$) 等高線圖上的搜索軌跡。與前述充滿密集陷阱 ($f2$) 或具備強烈邊界欺騙性 ($f3$) 的地形截然不同， $f4$ 是一個毫無干擾的平滑凸函數（呈現完美的同心圓），其核心目的是檢視演算法最純粹的「下坡收斂速度」與「局部開發」能力。觀察中圖的 PSO，在此無干擾環境下，其速度向量疊加與全局牽引的優勢得以完全釋放。其軌跡呈現極為直接、幾乎是兩點一線的直線俯衝，毫無懸念地精準命中靶心，從幾何角度完美印證了其在純淨地形下的極速尋優能力。右圖的 WOA 同樣展現了極高的收斂效率，其路徑呈現清晰的幾何特徵：以俐落的大步幅迅速向中心收攏，並透過簡潔的折線精確咬合全局最佳解

(紅星點)，完美詮釋了其包圍獵物機制的穩定性。反觀左圖的 Improved RO，雖然在此簡單地形中不至於受困，但其軌跡在逼近終點時，仍呈現出細微的盲目折返與試錯。這從空間視角再次證實，即使沒有陷阱阻撓，缺乏方向梯度記憶的隨機漫步，其收斂軌跡的平滑度與效率依然遠不及 PSO 與 WOA 這類群體智慧演算法。

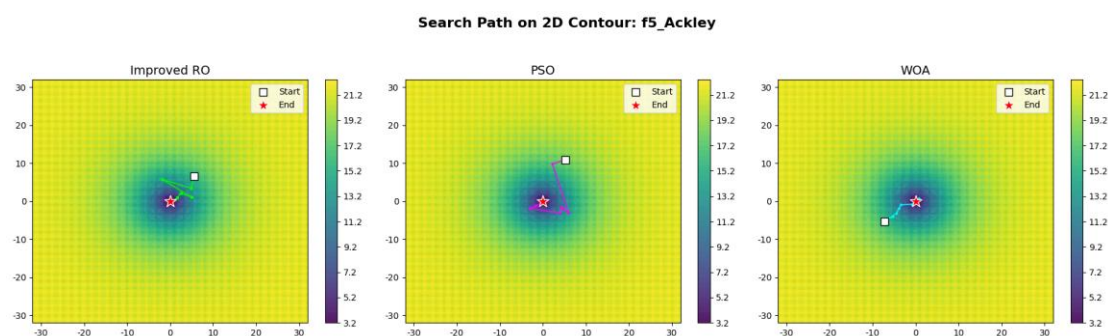


圖 2.3-5

圖 2.3-5 展示了維度 $D=2$ 時，三種演算法在具備「漣漪與漏斗」雙重特徵的 Ackley 函數 (f5) 等高線圖上的搜索軌跡。Ackley 的外圍是一大片佈滿細微局部極小值的平坦地形 (漣漪)，而正中心則隱藏著一個極度狹窄且陡峭的深谷 (漏斗)，這要求演算法必須同時具備大範圍的抗干擾能力與極高精度的微調能力。

觀察右圖的 WOA (青藍線)，其展現了極為俐落的尋優軌跡。它能以極高效率穿透外圍的干擾漣漪，並在接近中心深谷時，迅速切換為細膩的收攏步伐，精準且直接地「咬合」至全局最佳解 (紅星點)。這視覺化地證明了其在面對複雜地形時，具備極佳的探索與開發平衡能力。

中圖的 PSO (紫紅線) 則生動地展示了「速度慣性」的雙面刃效應。雖然它能憑藉大跨步迅速抵達中心區域，但在面對狹窄的深谷時，因慣性牽引過大而出現了明顯的「越界與折返」震盪現象。這種在谷口來回穿梭的軌跡，解釋了為何 PSO 在處理尖銳極小值時，往往難以瞬間精準停駐。

左圖的 Improved RO (綠線) 軌跡依然呈現破碎的隨機折線。雖然在低維度下它最終摸索到了中心，但其移動過程缺乏明確的梯度導向，在充滿細微陷阱的外圍平原上顯得跌跌撞撞，尋優效率明顯落後。總結來說，此圖從幾何維度完美確立了 WOA 在綜合型複雜函數中的壓倒性優勢。

參、結論與心得

3.1 綜合效能評比：WOA 的壓倒性優勢

本次最佳化演算法實驗針對五個具備不同地貌特徵的基準函數（ f_1 到 f_5 ），在跨維度（ $D=2$ 至 $D=30$ ）的嚴苛條件下，全面評估了 Improved RO、PSO 與 WOA 的尋找能力。綜合所有實驗數據與視覺化圖表，鯨魚最佳化演算法（WOA）展現了壓倒性的卓越性能，無論是在收斂精度、下降速度或抗干擾特性上，皆全面勝過另外兩者。

3.2 雙重視覺化證據支持與分析

1. 克服維度災難與早熟收斂（學習曲線證據）：

從圖 2.2 系列（學習曲線）**可以清晰觀察到，當維度提升至 $D=30$ 時，高維空間的複雜度會呈指數級放大。傳統的 PSO 極易因為粒子群過度依賴全局最佳解的牽引，導致多樣性過早喪失，在面對多峰陷阱（如 f_2 的 Rastrigin）或雙重漏斗地形（如 圖 2.2-5 f_5 Ackley）時，呈現致命的「水平停滯（早熟收斂）」。反觀 WOA，無論維度如何攀升，皆能以斷崖式的垂直軌跡迅速下墜，證明其隨機探索機制能有效強制群體跳脫局部極小值。

2. 精準的幾何開發能力（2D 軌跡證據）：

圖 2.3 系列（2D 搜索路徑圖）從空間幾何的角度解釋了 WOA 獲勝的底層邏輯。在 圖 2.3-1 (f_1) 與 圖 2.3-5 (f_5) 中，我們可以清楚對比出：Improved RO（綠線）流於盲目的隨機折返；PSO（紫紅線）雖具備動能，但在深谷前極易發生跨步過大的震盪（Overshooting）；而 WOA（青藍線）則完美發揮了「對數螺旋收縮」機制的優勢，以極其平滑、精準的折線完美咬合深谷底部的全局最佳解。

3.3 實驗心得

透過這次的實作與分析，我深刻體會到衡量一個最佳化演算法的優劣，關鍵在於其能否完美平衡「全局探索」與「局部開發」。PSO 雖然在毫無干擾的單峰地形（如 f_4 Sphere）中速度極快，但一旦面對具欺騙性或密佈陷阱的複雜地形，其剛性的速度更新公式便成為弱點。而 WOA 透過巧妙地模擬大自然中座頭鯨的氣泡網捕食行為，將數學模型與仿生學完美結合，達成了動態且具彈性的尋優策略。

此外，本次作業最大的收穫在於學會「如何用科學圖表說故事」。將冰冷的適應值數據轉化為多維度的學習曲線，並進一步結合等高線的 2D 軌跡圖，不僅讓我對演算法的底層運作有了透徹的「空間感」，更證明了雙重視覺化（數據趨勢與幾何路徑）是剖析、比較與驗證機器學習模型不可或缺的強大工具。

同時在本次作業的實作過程中，坦白說，我起初對進階演算法的程式碼撰寫感到有些吃力。然而，這也成為我轉換學習模式的絕佳契機。我以老師課堂講義的核心理論為基礎，並積極引入生成式 AI（Gemini）作為輔助工具進行協作。透過不斷地與 AI 探討程式碼邏輯、除錯，以及討論數據圖表的視覺化意義，我不僅克服了程式實作的門檻，更深刻體驗到「學習 AI 的同時，也與 AI 共學」的過程。這種人機協作的經驗，使我從單向的知識接收者，轉變為能主動設計實驗、解讀數據的探索者，這是我在本次課程中最寶貴的實務收穫。

肆、參考文獻

- [1] 李俊賢。《人工智慧簡論 (AI an overview_v2)》 [課堂講義]。國立中央大學資訊管理學系。
- [2] 李俊賢。《類神經網路基礎 (Basics of Artificial Neural Networks)》 [課堂講義]。國立中央大學資訊管理學系。
- [3] 李俊賢。《機器學習的啟發式演算法, Part-1 Metaheuristic Methods for Machine Learning (Part-1))》 [課堂講義]。國立中央大學資訊管理學系。
- [4] 李俊賢。《ML_HW_02_Assignment 作業說明》。國立中央大學資訊管理學系。